MidTerm Report

Giulia Beraldo, Geremia Paoletto

19th December 2025

1 Introduction

This document summarizes:

1. the main deviations from the original plan (Section 2);
2. the methodological details that were not specified in the proposal but were introduced during implementation (Section 3).

2 Deviations from the original plan

The overall pipeline (pruning → community detection → sampling) is unchanged. The main deviations from the proposal are the following:

- **Cluster for execution.** After initial tests, we realized that local machines were insufficient for processing road graphs with millions of nodes/edges and for running global procedures (e.g., community detection at scale and centrality measures). We therefore migrated execution to a department cluster node equipped with 240 GB of RAM, which allows us to keep the Northeast Italy graph fully in memory and to run global computations reliably.
- **Dataset: Northeast Italy instead of Germany.** To improve computational efficiency and shorten iteration cycles, we replaced the full Germany dataset with the Northeast Italy region (Veneto, Friuli-Venezia Giulia, Emilia-Romagna, Trentino–Alto Adige). The dataset contains 69,210,754 nodes and 7,929,412 ways, compared to 422,675,109 nodes and 506,112,013 ways for Germany. Despite the smaller size, the region preserves substantial geographic diversity (dense cities, flat rural areas, mountainous terrain), keeping the GeoGuessr-style task valid and non-trivial. We downloaded the dataset from `https://download.geofabrik.de/europe/italy/nord-est.html`.
- **Early topological simplification (degree-2 contraction).** During graph extraction, we added a simplification step that contracts degree-2 vertices on non-branching road segments to reduce graph size while preserving the connectivity skeleton. Concretely, if v has neighbours u and z and incident edges $\{u, v\}$ and $\{v, z\}$, we replace them with a single edge $\{u, z\}$ with weight

  $$w(u, z) \leftarrow w(u, v) + w(v, z).$$

  This preserves the length of the original polyline segment along contracted chains and substantially decreases memory usage and speeds up later global computations.
- **Urbanity score: PCA instead of heuristic weights / feature-space clustering.** We retain the same node-level feature set described in the proposal (degree, average edge length, road-type tag frequencies from OSM metadata, betweenness centrality (computed in an approximate way, using a function from `igraph`), local clustering coefficient), but replace heuristic feature weighting with a data-driven approach based on PCA. We standardize feature vectors x_v, compute PCA, and use the first principal component (PC1) as the urbanity score:

  $$s(v) = w_1^\top z_v,$$

  where z_v is the standardized feature vector and w_1 is the eigenvector associated with the largest eigenvalue of the covariance matrix. We fix the sign of w_1 to enforce a positive correlation between $s(v)$ and node degree, so that higher values consistently correspond to higher urban density. After computing $s(v)$, we run DBSCAN in one dimension on the set of values $\{s(v)\}$ to identify dense regions along the urbanity axis; nodes whose scores fall in the low-urbanity group are pruned.

- **Metrics used in the sampling step.** The goal of sampling is to select a small set of points that represent the discovered communities (ideally at least one point per community) and are as diverse as possible. The initial metrics considered (global summaries of Haversine distances and the Gini index) are only loosely aligned with these objectives. Since the problem is defined on a road network graph, we use **shortest-path distance on the graph** as the underlying distance $d(\cdot, \cdot)$ for both FFT/k-center-style reasoning and evaluation (weighted shortest path if edge weights are available, otherwise hop distance). We evaluate using:
  - *Community coverage:* for communities $C = \{C_1, \ldots, C_k\}$ and sampled set S,
    $$\text{coverage} = \frac{|\{\, i : C_i \cap S \neq \emptyset \,\}|}{k},$$
    and we summarize the distribution of $|C_i \cap S|$ to quantify how sampling mass is allocated across communities.
  - *Global coverage (k-center style):* for each node x, compute $d(x, S) = \min_{y \in S} d(x, y)$ and summarize $d(x, S)$ via $R = \max_x d(x, S)$ (worst-case radius), mean/median, and high percentiles.
  - *Diversity among sampled points:* compute pairwise distances $d(s_i, s_j)$ for $s_i, s_j \in S$ and report minimum separation $\min_{i \neq j} d(s_i, s_j)$ and summary statistics (mean/median/percentiles).
  - *Balance across communities:* using $n_i = |S \cap C_i|$, report an inequality indicator such as the coefficient of variation $CV = \sigma(n_i)/\mu(n_i)$ or an entropy-based score over the normalized counts.

  Midterm status: at this stage we specify the evaluation protocol and have the pipeline in place to compute these metrics; however, we do not yet report finalized metric values, as full runs across parameter settings and baselines are still ongoing. Quantitative results will be included in the final report.

3 Implementation details and parameter choices (not specified in the proposal)

- **Core graph analysis with `igraph`.** While the proposal already included using `leidenalg` via the `igraph` interface, during implementation we decided to keep the graph in `igraph` for the core analysis as well. This avoids expensive conversions and improves scalability on large graphs.
- **Choosing DBSCAN ε via the k-distance elbow.** When DBSCAN is used, ε is selected using the standard k-distance plot heuristic: we sort the distances to the k-th nearest neighbour and pick ε near the "elbow" (maximum curvature), where distances start increasing sharply.[1]
- **Leiden resolution parameter γ, refined tuning criterion.** The resolution parameter γ in the CPM objective controls the granularity of the detected communities. We tuned γ by sweeping a range of candidate values and, for each value, running Leiden multiple times with different random seeds. For each γ we evaluate:
  - the stability of partitions across runs (via normalized mutual information between clusterings) and
  - a domain-specific city-likeness score, measuring the fraction of nodes assigned to communities whose size falls within a plausible range for an urban area. We then select γ values exhibiting both high stability and high city-likeness, rather than simply maximizing the CPM objective value.

[1] `https://www.datacamp.com/tutorial/dbscan-clustering-algorithm`

4 Contribution of each member

The following contributions refer to the work completed up to the midterm deliverable. Percentages are approximate and reflect the time and implementation effort.

- Giulia Beraldo:
  - Implemented the main analysis pipeline: computation of the proposed graph-based scores/features and their integration into the pruning and clustering stages;
  - Implemented DBSCAN-based pruning and its data flow within the pipeline;
  - Implemented community detection using Leiden and performed fine-tuning of the resolution parameter γ (including multi-run stability checks);
  - Implemented both sampling strategies (community-based and FFT-based) and connected them to the evaluation pipeline.
  - Wrote most of the midterm report, including the description of deviations from the original proposal.
- Geremia Paoletto:
  - Proposed changes in the software stack/libraries (e.g., use of `igraph`) to improve scalability;
  - Explored the dataset and contributed to preprocessing decisions (initial analysis and feasibility checks);
  - Proposed the adoption of a PCA-based approach for defining an urbanity score and performed background research to support this choice;
  - Implemented the PCA-based urbanity score (feature standardization, PCA computation, score extraction and sign convention) and integrated it into the pruning workflow;
  - Ran the majority of experiments on the cluster and validated outputs (sanity checks on metrics and qualitative inspection);
  - Contributed to documenting the resulting methodological changes in the midterm report.

Task	**Giulia Beraldo**	**Geremia Paoletto**
Dataset exploration and preprocessing	Support	Lead
Urbanity score design (PCA approach)	Support (implementation)	Lead (proposal + research)
Library/tooling choices (e.g., `igraph`)	Lead (integration)	Lead (proposal and integration)
Leiden community detection + γ tuning	Lead	–
Sampling methods (community-based, FFT-based)	Lead	–
Running experiments on the cluster	Support	Lead
Midterm report writing and updates	Lead	Support (reported changes)
Estimated fraction of work	**50%**	**50%**

Table 1: Contribution summary up to the midterm deliverable.

5 AI usage

ChatGPT was used as a support tool for correcting the grammar of the report and for obtaining LaTeX phrasing suggestions.
Gemini was used for brainstorming and sanity-checking methodological options, in particular for:

- the suggestion to use the `igraph` library;
- pruning via DBSCAN;
- the definition and interpretation of the evaluation metrics;
- the use of PCA, including related definitions and concepts needed for the project.

All final technical decisions, implementations, and results were produced and verified by the authors.

Proposal stage. The proposal was jointly prepared by both members. AI tools were used for language polishing and for early brainstorming of alternative methods (e.g., possible clustering/community-detection approaches). Final choices were made by the authors.